

Risk-Prioritized Security Audit

of Zakura's Fork-Aware Header-Chain System

AI Auditor
example@example.org

August 16, 2026
Audit target: b47572d8f91e542cd1b4af05199fd7b75f69504d

AI-AUDIT WARNING

This security audit was performed by AI and this report was written by AI. AI analysis can be incomplete, mistaken, or overconfident. Bugs may have been missed, and every finding and remediation should be independently reviewed and tested by experienced Zakura maintainers and human security engineers. This document is not a security guarantee.

Methodology limitation. This \$300 risk-prioritized review used compact oracles, eight subsystem reviews, eight global attacks, two proofs, and eight validations. It did *not* complete the original 218 one-file-at-a-time checklist and must not be interpreted as exhaustive assurance over the approximately 88,000 lines inventoried for the original scope.

Severity	Confirmed findings
Critical	0
High	0
Medium	6
Low	1
Total	7

Contents

1	Introduction	3
1.1	Scope	3
1.2	Methodology and limitations	3
1.3	Risk summary	4
2	Threat Model	4
2.1	Assets and user expectations	4
2.2	Attackers and failure actors	4
2.3	Trust boundaries	5
2.4	Security invariants	5
2.5	Explicit limitation that is not a finding	5
3	Security & Privacy Findings	5
Issue A	Same-target scope advance strands stale body failures	6
Issue B	Deferred resource stall hot-loops the serialized state writer	6
Issue C	Queued header apply outlives timeout and shutdown revocation	7
Issue D	Native apply capability remains live after block-driver exit	8
Issue E	Fallible precommit rollback leaves uncommitted auxiliary state usable	8
Issue F	Circular recovery authority accepts coherently forged body invalidity	9
Issue G	Startup accepts unauthenticated finality history	10
4	Recommendations	11
4.1	Fix lifecycle and liveness defects first	11
4.2	Make resource refusal a scheduling event	11
4.3	Eliminate shared precommit mutation	11
4.4	Separate claimant rows from authority	11
4.5	Add adversarial composition tests	11
4.6	Improve operational detection	11
5	Future Work	11
5.1	Complete the original file-by-file checklist	11
5.2	Validate remaining clock and outcome hypotheses	12
5.3	Review authority constructibility under feature combinations	12
5.4	Expand invariant-gate independence	12
5.5	Fuzz crash, recovery, and event permutations	12
6	Conclusion	12
A	Audit Evidence Inventory	13
A.1	Compact oracles	13
A.2	Subsystem reviews	13
A.3	Global analyses	13
A.4	Proof attempts and validations	13
B	Dismissed Ownership Hypothesis	13
B.1	Unmatched body adoption reuses another request's owner	13
C	Severity Definitions	14

1 Introduction

This report presents a risk-prioritized security review of Zakura’s fork-aware header-chain system. Zakura is a Rust implementation of a Zcash full node. The audited subsystem plans synchronous, invariant-checked transitions over a bounded header DAG and coordinates those transitions with header sync, block sync, durable state, publication, recovery, and node drivers.

The review identified seven confirmed findings: six *Medium* issues and one *Low* issue. No Critical or High issue was confirmed. The Medium findings fall into three clusters:

- remotely initiated synchronization and resource-liveness failures;
- capability, cancellation, and driver lifecycle inconsistencies; and
- recovery or atomicity failures requiring local storage faults, privileged corruption, or compound failure conditions.

None of the confirmed findings, by itself, demonstrates that an ordinary network peer can cause Zakura to verify or finalize an arbitrary fork. Several findings can nevertheless stop synchronization, suppress mandatory body invalidity evidence, falsely penalize peers, consume a CPU core, or cause runtime and durable state to disagree.

1.1 Scope

The target was the clean Zakura tree at commit:

b47572d8f91e542cd1b4af05199fd7b75f69504d

The normative oracle was `docs/specs/fork-aware-header-chain-engine.md`, with Zcash consensus rules taking precedence over the local `LC-*` requirements [3, 2]. The primary implementation scope was:

- `crates/zakura-header-chain/src/transition/`;
- the header-sync and block-sync implementations in `crates/zakura-network/src/zakura/`;
- state construction, commit, recovery, and publication paths in `crates/zakura-state/src/service/`;
- the header and block drivers and lifecycle coordinator under `crates/zakurad/src/commands/start/zakura/`; and
- supporting graph, work-authority, validation, and service-lifecycle types needed to establish end-to-end reachability.

The review considered tests as evidence, not as authority. It followed cross-component interactions where an outcome could violate transition atomicity, authority or freshness, body-evidence separation, finality, retention bounds, recovery independence, publication ordering, or exactly-once lifecycle handling.

Wallet functionality, unrelated RPC and peer-management defects, transaction validation outside these paths, the legacy Zcash P2P stack, deployment configuration, and performance issues without security, safety, or liveness impact were out of scope.

1.2 Methodology and limitations

The audit used invariant-centric threat modeling [1], three compact oracles (authority boundaries, event/outcome behavior, and the `LC-*` requirement map), eight subsystem reviews, eight global adversarial analyses, two attempted proofs, and eight issue validations. The subsystem reviews covered authority/freshness, planner atomicity, the invariant gate, recovery, header sync, block sync, state commit, and driver orchestration. Global reviews covered stale completion ownership, body evidence, finality/recovery, VCT authentication, lifecycle split brain, resource-stall denial of service, recovery-evidence independence, and counters/clocks/deferral.

The two proof attempts examined transition atomicity and exact branch authority. They were useful precisely because concrete counterexamples broke the intended properties at component boundaries. Seven candidates were confirmed and one ownership hypothesis was dismissed after end-to-end validation.

Non-exhaustive review. The original process inventoried 218 audit-relevant files for one-file-at-a-time review. That checklist remained incomplete. This report reflects a \$300 risk-prioritized review, not complete line-by-line coverage, formal verification, dynamic exploit development, or exhaustive testing of every event ordering. Absence of a finding in an unreviewed or deprioritized path is not evidence of security.

1.3 Risk summary

ID	Finding	Severity	Primary impact
A	Same-target scope advance strands stale body failures	<i>Medium</i>	Process-lifetime body-sync stall; invalidity evidence loss
B	Deferred resource stall hot-loops the serialized writer	<i>Medium</i>	CPU, lock, and state-throughput denial of service
C	Queued header apply outlives timeout and shutdown revocation	<i>Medium</i>	Late mutation, false peer penalties, duplicate work
D	Native apply capability remains live after block-driver exit	<i>Medium</i>	Persistent verified-sync outage until restart
E	Fallible precommit rollback leaves staged state usable	<i>Medium</i>	In-memory/durable split authority under storage faults
F	Circular body-evidence recovery authority accepts forged invalidity	<i>Medium</i>	Persistent valid-branch suppression after corruption
G	Startup accepts unauthenticated finality history	<i>Low</i>	Forensic and migration provenance loss

Table 1: Confirmed findings, ordered primarily by practical remote reachability and operational impact, then by privileged-failure risk.

2 Threat Model

2.1 Assets and user expectations

The assets protected by this subsystem are:

- the deterministic selected-header view;
- the independently authenticated fully verified view;
- monotone and provenance-authenticated finalized history;
- coherent graph rows, indexes, metadata, counters, durable storage, in-memory state, and published snapshots;
- durable invalidity and body-evidence conclusions;
- bounded CPU, memory, storage, queues, retries, and retained paths;
- correct attribution of peer success and failure; and
- deterministic, fail-closed restart and recovery.

Users should be able to rely on header acceptance not implying body validity, body availability not changing fork choice, stale work having no effect, and failed transitions not leaving usable state. They should also expect a live service not to advertise apply capacity without a consumer.

2.2 Attackers and failure actors

The review considered malicious or faulty peers, eclipse attackers, adversarial block producers, buggy or compromised adjacent components, privileged storage writers and migration tools, storage faults, crashes, scheduler-controlled orderings, operator lifecycle actions, and wall-clock changes. Network attackers control timing, omission, duplication, reordering, headers, bodies, advertised ranges, and peer churn, but do not automatically possess state capabilities or full-verifier authority.

2.3 Trust boundaries

The principal trust boundaries are:

1. network input to sync scheduling and framing;
2. sync registration to exact transition admission authority;
3. sealed preparation to commit against current branch and generation;
4. full-state verification to body, VCT, and integrated-finality evidence;
5. planner effects to the independent invariant gate;
6. in-memory plans to durable commit, installation, and publication;
7. durable rows to independently authenticated recovery facts;
8. published snapshots to driver ownership and terminal outcomes; and
9. configuration and release data to trusted pins and network identity.

Public hashes, generations, branch IDs, evidence digests, request IDs, and source IDs are correlation values, not possession authority. The design relies on typed capabilities, live registrations, authenticated canonical state, and full-state receipts to turn those values into authorized conclusions.

2.4 Security invariants

The review organized the specification into ten user-facing invariants:

1. failed planning or commit leaves no usable partial effect;
2. eligible fork choice is deterministic and uses checked accumulated work;
3. header and verified frontiers remain independent;
4. every state, scheduling, scoring, and coverage effect has exact fresh authority;
5. finality is monotone, mode-specific, and provenance-authenticated;
6. invalidity is permanent where required and reason-scoped;
7. auxiliary data remains isolated until exact VCT authentication;
8. hostile operation remains bounded and resource pressure is recoverable;
9. recovery rejects authoritative corruption and repairs only from independent facts; and
10. driver exit, cancellation, fallback, and restart revoke or transfer capabilities coherently.

2.5 Explicit limitation that is not a finding

A headers-only client cannot prove body or state validity from headers. Its 1,000-block-deep finality is an explicit local trust decision. This impossibility boundary is documented and was not treated as a vulnerability. Integrated mode also depends on authentic conclusions from its consensus, full-state, operator, configuration, clock, and release-trust interfaces.

3 Security & Privacy Findings

Ratings combine user impact with realistic exploitability. *Medium* denotes moderate impact or a serious issue requiring special circumstances. *Low* denotes limited impact or unlikely exploitation. The findings below are ordered by risk and practical trigger conditions; this ordering does not change the assigned severity.

Issue A Same-target scope advance strands stale body failures

Severity: *Medium*

Affected code. `crates/zakura-network/src/zakura/block_sync/reactor.rs`, `sequencer_task.rs`, `work_queue.rs`, and the `state/driver body-commit` path.

Description. A verified body commit advances `verified_generation` and publishes the new committed snapshot before a later concurrent block-apply future returns. When the selected header target is unchanged, block sync intentionally preserves already downloaded bodies and their old-scope queue claims. Every later non-verified completion is nevertheless judged against the new generation and classified stale.

The stale path removes the applying entry but does not remove or rebind the `WorkQueue` claim, roll back the download floor, trigger scheduling, record retry/unavailability, persist deterministic invalidity, or score the supplier:

```

1 // block_sync/sequencer_task.rs:693-729
2 if applying_owner != owner
3     || applying_source != source
4     || applying_hash != hash
5     || applying_token != token
6 {
7     // detached completion
8 }
9 if !semantic_current {
10     let _ = self.sequencer.remove_applying(height);
11     self.sequencer
12         .finish_submission(owner, source, token, height, hash);
13     return (false, false);
14 }
```

The queue then refuses to reacquire the stranded height:

```

1 // block_sync/work_queue.rs:133-169
2 for (height, hash, size) in items {
3     if height <= inner.floor
4         || inner.pending.contains_key(&height)
5         || inner.in_flight.contains_key(&height)
6     {
7         continue;
8     }
9     // insert pending item
10 }
```

Attack scenario. Two contiguous bodies are submitted concurrently under generation n . The earlier valid body commits, advances the verified frontier, and publishes generation $n + 1$ while preserving the same selected target. A malicious supplier times the later body to complete afterward as a payload mismatch, retryable failure, or consensus-invalid body. Exact request identity still matches, but generation freshness does not. The stale branch discards the semantic result while leaving the height claimed.

Impact. Body synchronization can remain stuck for the process lifetime until restart or a destructive scope reset. Payload mismatch is enough for the remote liveness impact. If the exact commitment-matching body is consensus-invalid, the permanent tombstone and inherited branch ineligibility are also suppressed until the body is downloaded and rejected again. The issue does not authorize an arbitrary verified frontier.

Recommendation. Choose one ownership model for same-target advancement. Either atomically rebind preserved bodies and all queue claims to the new authority, or retire the exact old claim and roll the floor back on stale non-verified completion. Authenticate deterministic invalidity using the stored exact applying identity rather than discarding it solely because verified progress advanced the generation. Add permutation tests with a nonempty queue for completion before and after watch processing.

Issue B Deferred resource stall hot-loops the serialized state writer

Severity: *Medium*

Affected code. `crates/zakura-state/src/service/write.rs`, `crates/zakura-state/src/service/finalized_state/header_chain.rs`, and `crates/zakura-header-chain/src/transition/planner/retention.rs`.

Description. A due deferred header can become eligible when all retention victims are protected by active retained-path leases. The transition engine correctly returns `ResourceStalled` and leaves the due row unchanged. Deferred maintenance discards the typed result:

```

1 // zakura-state/src/service/write.rs:621-629
2 fn apply_deferred_reevaluation(&self) -> Result<(), HeaderChainStoreError> {
3     let _ = self.runtime.apply(
4         TransitionRequest {
5             expected_version: self.runtime.publisher().snapshot().state_version,
6             event: TransitionEvent::ReevaluateDeferred,
7         },
8         &self.context(),
9     )?;
10    Ok(())
11 }

```

The scheduler immediately reloads the same already-due deadline and retries without sleeping:

```

1 // zakura-state/src/service/write.rs:1699-1729
2 loop {
3     match receiver.try_recv() {
4         Ok(message) => return Ok(Some(message)),
5         Err(TryRecvError::Disconnected) => return Ok(None),
6         Err(TryRecvError::Empty) => {}
7     }
8     let Some(deadline) = maintenance.earliest_deferred()? else {
9         return Ok(receiver.blocking_recv());
10    };
11    if deadline <= maintenance.now() {
12        maintenance.reevaluate_deferred()?;
13        continue;
14    }
15 }

```

Attack scenario. The graph has ten eligible tips and one deferred sibling. Ten peer sessions lease paths protecting nine non-selected valid tips and the deferred tip; `header_best` protects the last valid tip. Promotion would create eleven eligible tips, but every victim is protected, so retention refuses. Peers renew the 30-second leases with low-rate continuation reads while the node performs an unbounded number of local retries.

Impact. The loop can consume a CPU core and repeatedly perform RocksDB reads, graph scans, retention work, invariant checks, and lock acquisition on the serialized writer. Queued messages are checked between iterations, so absolute starvation was not established, and only the first stall changes durable alarm state. Nevertheless, state throughput and synchronization can be materially degraded. The topology requires many retained valid forks, limiting exploitability.

Recommendation. Preserve and handle every `ApplyResult`. On `ResourceStalled`, wait for a lease/resource-change notification or use bounded exponential backoff. Never immediately reread an unchanged due row. Test one alarm publication, bounded retry frequency, queue service, lease expiry, and eventual promotion.

Issue C Queued header apply outlives timeout and shutdown revocation

Severity: *Medium*

Affected code. Header-sync reactor and driver, the state-service queue, and the serialized header writer.

Description. After a valid target passes the live registration gate, state synchronously queues `ApplyHeaderChainInsert`. If the reactor's generic request deadline expires during local state delay, the reactor removes ownership, releases capacity, and penalizes the peer. Shutdown similarly retires reactor work. Dropping the response future does not remove the already queued message:

```

1 // zakura-state/src/service.rs:1076-1093
2 let (rsp_tx, rsp_rx) = oneshot::channel();
3 let Some(sender) = &self.block_write_sender.non_finalized else {
4     let _ = rsp_tx.send(Err(HeaderChainStoreError::Uninitialized));
5     return rsp_rx;
6 };
7 if let Err(SendError(message)) =
8     sender.send(NonFinalizedWriteMessage::ApplyHeaderChainInsert {
9         prepared,

```



```

10     rsp_tx,
11   })
12   {
13     // send failure handling
14   }
15   rsp_rx

```

The writer consumes a one-shot permit but does not consult reactor revocation. If branch coordinates remain current, it can commit and publish after timeout or shutdown.

Attack scenario. A peer supplies a valid target. The reactor admits it, the driver queues it, and local writer delay exceeds the deadline. The reactor records a terminal timeout and charges the peer. The writer later commits the same valid target. The response is dropped or ignored because the active slot is gone, so completed-target coverage is not marked and the penalty is not reversed.

Impact. Consensus and exact-branch checks still constrain the committed headers. However, a retired component causes durable mutation and publication, an honest supplier can be falsely penalized or disconnected, and the target can be rediscovered and downloaded again. Shutdown/restart handoff loses exactly-once terminal interpretation.

Recommendation. Once state accepts an apply, disarm the peer/network timeout and retain the registration until state returns. Separate peer responsiveness deadlines from local-health watchdogs. Drain accepted applies during shutdown or add a revocation epoch checked immediately before commit. Regression tests should cross enqueue/dequeue/commit barriers with timeout, shutdown, generation change, and restart.

Issue D Native apply capability remains live after block-driver exit

Severity: *Medium*

Affected code. `crates/zakurad/src/commands/start/zakura/block_sync_driver.rs`, `coordinator.rs`, endpoint task supervision, and the block reactor and sequencer.

Description. The block action driver intentionally exits if it cannot durably persist current deterministic body invalidity. This local fail-close behavior is appropriate. The shared coordinator, however, remains in `ApplyPhase::Native`. Shutdown requests only operation cancellation:

```

1 // zakurad/src/commands/start/zakura/coordinator.rs:252-255
2 pub(crate) fn request_apply_shutdown(&self) {
3     self.request_operation_cancellation();
4 }

```

Service demand therefore continues advertising `ServingAndApplying`, while the sole action receiver has been dropped. The driver task is retained but not supervised or restarted.

Attack scenario. A malicious producer creates a selected commitment-matching body that fails consensus verification. Persistence of the resulting invalidity then encounters a local state error, timeout, resource stall, unexpected response, or repeated stale race. The driver drains accepted work and exits. The coordinator remains Native, the reactor stays live, and subsequent operations meet a permanently closed action channel.

Impact. A Zakura-only node can appear reachable while verified body sync remains stalled until process restart. Dual-stack fallback can temporarily progress, but dropping its lease resumes Native without recreating the consumer. No Native/legacy simultaneous commit authority or invalid durable commit was established. The trigger requires both adversarial invalid body conditions and a local persistence failure.

Recommendation. Make every driver exit atomically revoke its capability: transition the coordinator to `Failed` and publish serving-only demand, terminate the endpoint/process, or establish a replacement consumer before Native can be advertised. Supervise the task and prohibit `ResumeNative` without a live driver for the new epoch.

Issue E Fallible precommit rollback leaves uncommitted auxiliary state usable

Severity: *Medium*

Affected code. `crates/zakura-state/src/service/finalized_state/header_chain.rs` and the finalized writer.

Description. The combined VCT auxiliary-authentication/checkpoint path installs the first transition into the shared engine before the combined RocksDB batch commits. If a later operation fails, compensation reloads the engine from durable rows. That reload can also fail. Its failure branch logs and returns without replacing or poisoning the staged engine:

```

1 // zakura-state/.../header_chain.rs:413-431
2 fn restore_transition_engine_after_staging_error(
3     store: &HeaderChainStore,
4     engine: &mut HeaderChainEngine,
5     original: HeaderChainStoreError,
6 ) -> HeaderChainStoreError {
7     match load_transition_engine(store) {
8         Ok(restored) => {
9             *engine = restored;
10            original
11        }
12        Err(reload) => {
13            tracing::error!(?original, ?reload,
14                "failed to restore the durable header engine");
15            reload
16        }
17    }
18 }

```

The finalized writer resets its queue and continues. Readers and later planners can therefore use auxiliary state that was neither committed nor published.

Attack scenario. A valid checkpoint auxiliary transition is staged. The combined database write fails due to a local storage fault, and durable reconstruction also fails due to a read fault or corrupt row. Locks are released normally; the shared engine retains the staged transition while the database and published watch remain at the before-state.

Impact. The runtime has split authority. Later plans can derive from an auxiliary fact or version absent from durable state, causing repeated coherence failures, stuck writers, or fail-closed restart. A peer alone cannot trigger the finding, and no immediate wrong-fork publication was shown. Exploitation requires compound local failures expressly included in the audit threat model.

Recommendation. Plan the composite operation on an isolated engine or overlay and install only after the one durable batch succeeds. If shared staging remains, keep an infallible before-image. A failed rollback must poison/retire the runtime and terminate the writer before any reader can acquire the staged engine.

Issue F Circular recovery authority accepts coherently forged body invalidity

Severity: *Medium*

Affected code. State header-chain disk encoding and `crates/zakura-header-chain/src/transition/recovery/`.

Description. Recovery treats `HEADER_BODY_EVIDENCE_AUTHORITY` as independent full-state authority for durable Verified and ConsensusInvalid states. The only production encoder constructs that row by copying the exact public fields from the header node that it later authenticates. Attestation is only equality:

```

1 // disk_format/header_chain_values.rs:393-453
2 pub fn attests_to_body_validation_state(
3     &self,
4     header_hash: block::Hash,
5     body_validation_state: &BodyValidationState,
6 ) -> bool {
7     match (self, body_validation_state) {
8         (Self::Verified { hash, evidence },
9          BodyValidationState::Verified { evidence: claimed }) =>
10         *hash == header_hash && evidence == claimed,
11         (Self::ConsensusInvalid(authority),
12          BodyValidationState::ConsensusInvalid { evidence, rule }) =>
13         authority.hash == header_hash
14         && authority.evidence == *evidence
15         && authority.rule == *rule,
16         _ => false,

```

```

17   }
18 }

```

These rows share provenance. Equality between them does not prove an independent verifier or full-state conclusion.

Attack scenario. A privileged corrupt writer, faulty migration/restore, or correlated durable corruption inserts a consensus-invalid tombstone and a matching copied authority row for a valid public hash. Recovery accepts them. A retained node can also be changed to the matching body state. A no-node tombstone survives pruning rules and later marks the valid block invalid when it is inserted.

Impact. The node can permanently suppress a valid present or future branch across restart and remain on a weaker or stale eligible chain. A more constrained forged-Verified side-path variant may affect later verified-path reselection. This is not remotely reachable through normal protocol requests; raw database write or correlated logical corruption is required. Ordinary crash atomicity and the supported rollback path do not create the forged state.

Recommendation. Persist verifier-owned body conclusions in an independent full-state namespace that the header change-set encoder cannot mint. Bind each receipt to network, hash, rule, evidence, verifier domain/version, transition identity, and canonical/full-state location as applicable. Recovery should enumerate those receipts and reject claimant-only or receipt-only conclusions. Add coherent multi-row corruption tests, including no-node tombstones.

Issue G Startup accepts unauthenticated finality history

Severity: *Low*

Affected code. `crates/zakura-header-chain/src/transition/recovery/audit/authoritative_rows.rs` and state finality-history storage/migration.

Description. Recovery checks finality-record structure but not every provenance claim. Integrated records are accepted based on the `FullState` enum variant without consulting an independent receipt. Headers-only records check the numeric 1,000-block depth but not that the recorded finalized frontier is the ancestor of the exact selected-tip hash:

```

1 // recovery/audit/authoritative_rows.rs:176-205
2 match (metadata.mode, metadata.headers_only_migration_epoch, record.source) {
3     (EngineMode::Integrated, None, FinalitySource::FullState { .. }) => true,
4     (EngineMode::HeadersOnly, None,
5      FinalitySource::HeadersOnlyDepth { selected_tip }) => {
6         record.current.height > record.previous.height
7         && selected_tip.height.0
8         .saturating_sub(record.current.height.0)
9         == config.limits.local_finality_depth.get()
10    }
11    _ => false,
12 }

```

The live planner and invariant gate do perform exact ancestry checks; the gap is specific to startup and migration.

Attack scenario. A privileged writer or correlated corruption changes an integrated evidence ID or substitutes an unrelated same-height headers-only tip while preserving record continuity, epochs, and the genuine terminal frontier. Recovery accepts the forged history. A later explicit migration can relabel the weakly audited headers-only records as migrated history.

Impact. Operators can lose trustworthy forensic and migration provenance for irreversible finality decisions. The current finalized tip remains protected by a separate authenticated canonical-height lookup, so the issue alone does not move active finality to an arbitrary hash or let a peer finalize a branch. This containment and the privileged-corruption precondition support Low severity.

Recommendation. Persist integrated finalization receipts independently and bind every epoch, previous/current frontier, mode, network, and evidence identity. For headers-only history, verify an authenticated exact selected path and ancestor proof at startup. Migration must reject history whose original proof cannot be authenticated.

4 Recommendations

4.1 Fix lifecycle and liveness defects first

Prioritize Issues Issue A–Issue D. They are the most operationally reachable and compose with network timing and ordinary service load. Introduce explicit lifecycle states for queued, admitted, committing, completed, canceled, and transferred work. A terminal outcome should be interpreted exactly once by a component that still owns the operation, and peer deadlines should stop before purely local state delay.

4.2 Make resource refusal a scheduling event

`ResourceStalled`, `Stale`, `NoChange`, and `Committed` are semantically distinct. Avoid wildcard result handling. Every caller should document and test how each result changes retry timing, coverage, alarms, peer attribution, and wakeups. Resource availability should have an explicit notification path; bounded backoff is a fallback, not the primary coordination mechanism.

4.3 Eliminate shared precommit mutation

Use immutable plans, isolated overlays, or copy-on-write snapshots for compound transactions. The shared engine should only advance after the exact durable batch succeeds. Where rollback exists, it must be in-memory and infallible, or failure must permanently retire the runtime before locks are released.

4.4 Separate claimant rows from authority

Recovery authority must come from an independently owned namespace and writer. Do not authenticate a conclusion by comparing it with a second serialization derived from that conclusion. Domain-separate and bind receipts to network, mode, branch, generation, event, verifier version, and durable state location.

4.5 Add adversarial composition tests

Existing tests often establish each half of a failure independently. Add barrier-controlled end-to-end tests for:

- generation advance racing every body outcome with a nonempty queue;
- header enqueue racing timeout, shutdown, commit, and response delivery;
- driver exit observed through shared coordinator demand and channel state;
- due deferral under fully protected retention pressure;
- database-write failure followed by rollback/reload failure; and
- coherent multi-row corruption that preserves superficial equality.

4.6 Improve operational detection

Expose stable alarms for applying service without a consumer, queue claims without a live owner, repeated no-change resource stalls, durable/published version divergence, and driver task exit. Metrics should support alerts, but correctness must not depend on an operator noticing them.

5 Future Work

5.1 Complete the original file-by-file checklist

The highest-priority future task is completion of the original 218-file, one-file-at-a-time review. The current risk-prioritized method found meaningful cross-component defects but cannot establish that lower-priority files contain no additional vulnerabilities.

5.2 Validate remaining clock and outcome hypotheses

The global clock/deferral review identified low-severity candidates around wall-clock rollback at restart and forward clock correction failing to wake an armed deferral. These were not among the eight validated hypotheses and are not findings in this report. They should receive dynamic validation. Availability and supplier-discovery state writes also deserve complete four-way `ApplyResult` tests.

5.3 Review authority constructibility under feature combinations

Review release builds that enable testing/property features, especially any feature exposing broad evidence-sealing constructors. Confirm that no downstream crate can mint production authority through a normal Cargo feature. Also split broad capabilities by purpose to make future confused-deputy errors harder to introduce.

5.4 Expand invariant-gate independence

The compact reviews found that some freshness, ownership, VCT, finality, and preparation properties are checked primarily by adapters or planner handlers. Where feasible, make the final gate independently recompute security-critical claims rather than only checking self-consistency of planner output.

5.5 Fuzz crash, recovery, and event permutations

Extend the crash harness to compound failures, coherent corruption, watch coalescing, lifecycle restart, and every ordering of insert, body result, finality, reset, retention pressure, deferred wakeup, and driver cancellation. Use a small executable reference model for exact ownership and event/outcome translation.

6 Conclusion

This risk-prioritized review confirmed seven findings: six Medium and one Low. The most practical defects are synchronization and lifecycle failures at the boundaries between network reactors, drivers, state queues, and typed transition outcomes. A malicious peer can contribute to process-lifetime body-sync stalls, and retained-path leases can sustain a local hot loop under a demanding fork topology. Valid header work can also commit after its owner has timed out or shut down.

The remaining findings expose important fail-closed boundaries under storage faults or privileged durable corruption. Shared precommit staging can survive a failed rollback, body-recovery “authority” is circular, and finality-history provenance is incompletely authenticated. These conditions are less directly exploitable, but they matter because recovery and storage corruption were explicitly part of the threat model.

The transition engine contains meaningful defenses: exact branch and generation checks, checked planning, an independent invariant gate, atomic RocksDB batches, separation of body and header conclusions, and authenticated current finality. Those defenses materially limit every confirmed issue. They do not replace the need to repair lifecycle coordination, resource-stall scheduling, and independent recovery evidence.

This report must not be read as exhaustive assurance. It is an AI-generated report from a \$300 risk-prioritized review, and the original 218-file linear checklist was not completed. Human review, remediation testing, and broader follow-up audit work remain necessary.

References

- [1] Invariant-Centric Threat Modeling. <https://github.com/defuse/ictm>.
- [2] Electric Coin Company and Zcash Foundation. Zcash Protocol Specification. <https://zips.z.cash/protocol/protocol.pdf>.
- [3] Zakura Project. Fork-Aware Header-Chain Transition Engine Specification, 2026. Audit target documentation at commit b47572d8f91e542cd1b4af05199fd7b75f69504d.

A Audit Evidence Inventory

A.1 Compact oracles

- `audit-state/oracles/authority-boundaries.md`
- `audit-state/oracles/event-outcome-matrix.md`
- `audit-state/oracles/lc-requirement-map.md`

A.2 Subsystem reviews

The eight subsystem reviews addressed authority/freshness, planner atomicity, the invariant gate, recovery, header sync, block sync, state commit, and driver orchestration. They supplied component-local evidence and coverage gaps that were then tested against the global and proof analyses.

A.3 Global analyses

The eight global analyses addressed stale completion ownership, body-evidence lifecycle, finality/recovery, VCT authentication, driver lifecycle split brain, resource-stall denial of service, recovery-evidence independence, and counters/clocks/deferral.

A.4 Proof attempts and validations

The transition-atomicity proof produced the precommit rollback counterexample. The exact-branch-authority proof produced or reinforced the stale body failure, late header commit, and unmatched-body ownership hypotheses. Eight validations then resolved the seven confirmed findings and one invalid hypothesis.

B Dismissed Ownership Hypothesis

B.1 Unmatched body adoption reuses another request's owner

The audit explicitly validated and dismissed the hypothesis `unmatched-body-borrows-owner.md`. When a different authenticated peer supplies an exact currently wanted body, the unmatched path reuses the existing queue item's owner while preserving the actual supplier as a distinct source. The tuple was not present in the original transport request registry, but the operation is an intentional first-completion-wins transfer:

```

1 // block_sync/peer_routine.rs:1660-1732
2 if self.work.hash_for_height(height) != Some(hash) {
3     return false;
4 }
5 let Some(owner) = self.work.owner_for_height(height) else {
6     return false;
7 };
8 let Some(estimate) = self
9     .work
10    .release_active_reserved_height_for_owner(owner, height)
11 else {
12     return false;
13 };

```

Adoption requires the exact wanted hash, a live queue owner and byte reservation, and an authenticated peer range covering the height. Completion is later bound to exact owner, actual source, token, height, and hash. Full-state verification still checks commitments and consensus. Invalid bodies are attributed to the delivering peer, not the displaced request owner.

No unauthorized durable state, wrong-peer attribution, stale capability, or fork-safety impact was established. The hypothesis is therefore dismissed and is *not* included in the finding count. Documenting adoption as an atomic ownership transfer and replacing the temporary registration check with an explicit adoption registry would improve clarity. A separate denial-of-service review of unsolicited first completions may still be worthwhile, but that would be a different hypothesis.

C Severity Definitions

Critical. Easily exploitable with devastating consequences for many users.

High. Likely to have severe impact on many users.

Medium. Moderate impact, limited reach, or serious impact requiring special circumstances.

Low. Limited impact or exploitation unlikely in practice.